



Desenvolvimento Web

Profª Sabrina B. Moreira

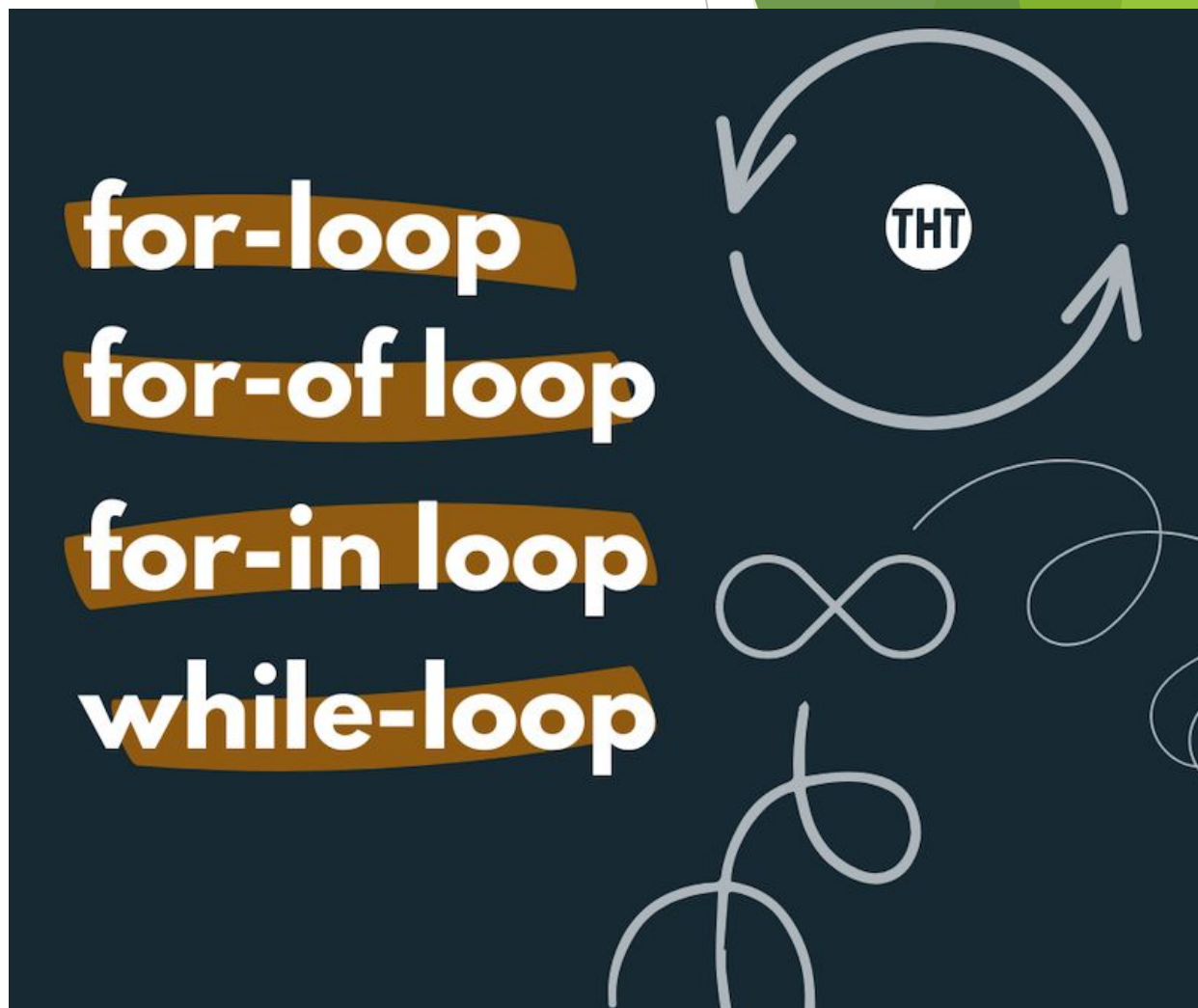


JavaScript

Aula 18.1: loops, arrays, objects e dates

JS: laços de repetição (loops)

- ❖ Os laços de repetição, também conhecidos como *loops*, são estruturas utilizadas para executar um mesmo **bloco de código** várias vezes de forma automática.



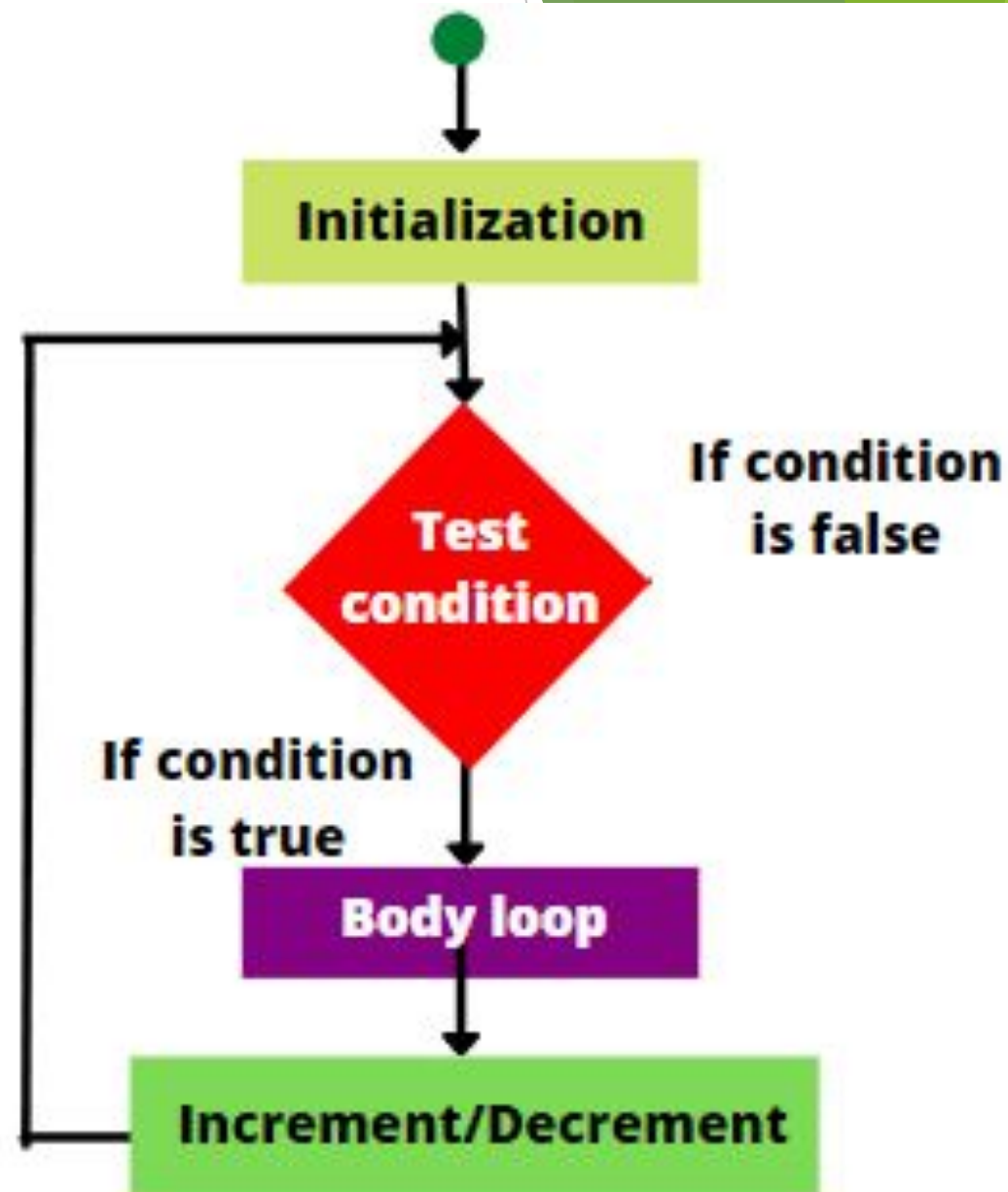
JS: laços de repetição (loops)

- ❖ Eles são úteis quando precisamos **realizar tarefas repetitivas**, como exibir uma sequência de números, percorrer listas de dados, validar entradas do usuário ou executar uma ação até que **determinada condição** seja atendida.



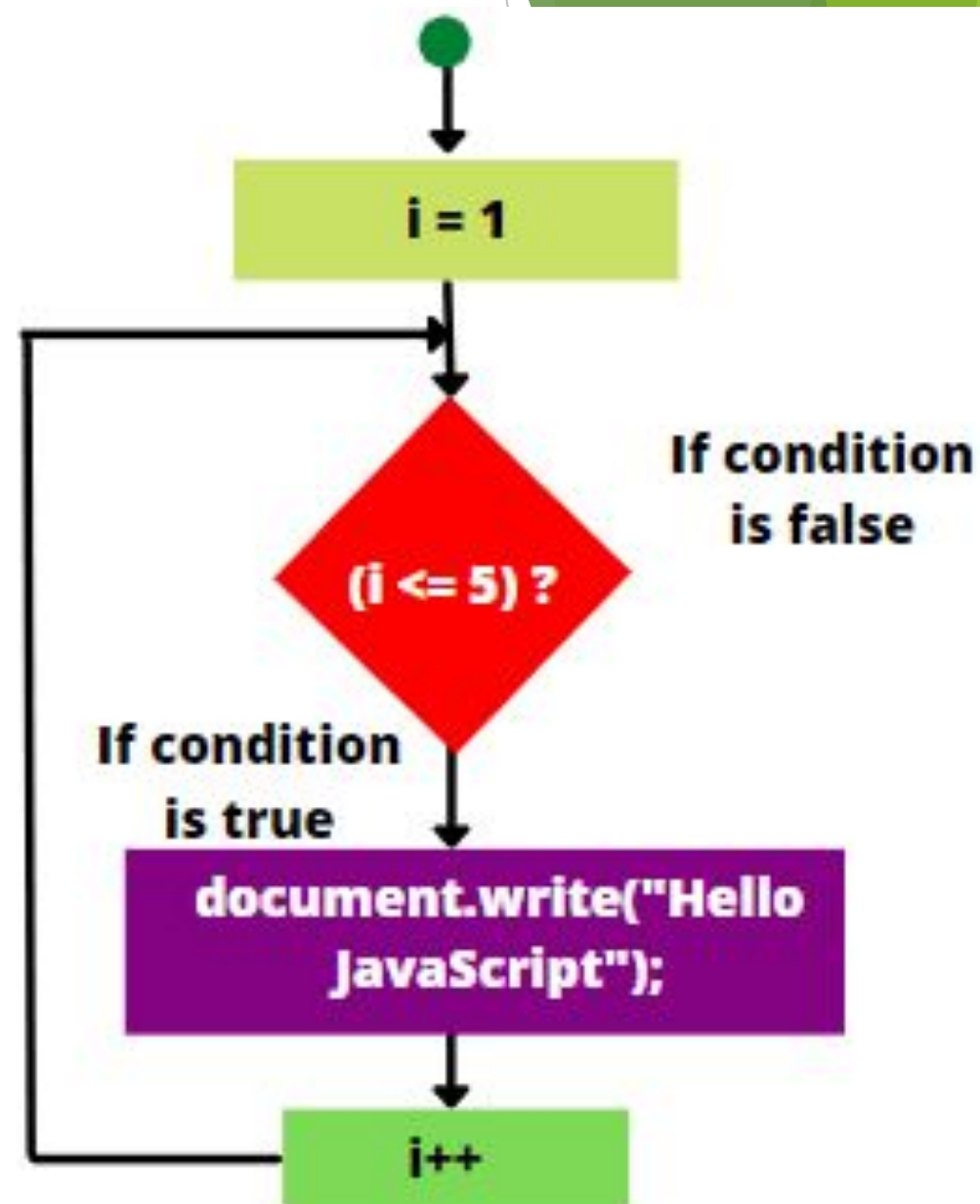
JS: laços de repetição (loops)

- ❖ Em vez de escrever o mesmo código diversas vezes, utilizamos um loop para controlar quantas vezes ele será executado, tornando o programa mais organizado, reutilizável e fácil de manter.



JS: laços de repetição (loops)

- ❖ No JavaScript, os principais laços de repetição são *while*, *do...while* e *for*.
- ❖ Cada um indicado para situações específicas.



JS: variável de controle

- ❖ A variável de controle é utilizada para acompanhar o progresso das repetições dentro de um laço.
- ❖ Ela é inicializada antes do loop, utilizada na condição de repetição e atualizada a cada execução.

```
for (let k = 1; k <= 5; k++) {  
  console.log("k: " + k);  
}
```


JS: variável de controle

- ❖ Seu uso correto é fundamental para evitar comportamentos inesperados, como repetições excessivas ou loops infinitos.

```
let i = 1;

while (i <= 5) {
  console.log("i: " + i);
  i++;
}
```

Fonte: autora, 2026

JS: variável de controle

- ❖ A alteração de valor da variável de controle pode acontecer tanto aumentando o seu valor, quanto diminuindo, e também, em janelas de valores.

```
for (let k = 5; k >= 1; k--) {  
  console.log("k: " + k);  
}
```

k: 5
k: 4
k: 3
k: 2
k: 1

```
for (let k = 1; k <= 5; k += 2) {  
  console.log("k: " + k);  
}
```

k: 1
k: 3
k: 5

JS: variável de controle

- ❖ A atualização dessa variável pode acontecer pelo operador de atribuição com incremento/decremento ou realizando a atribuição simples da variável.

```
for (let k = 1; k <= 5; k++) {  
  console.log("k: " + k);  
}
```

k: 1

k: 2

k: 3

k: 4

k: 5

```
for (let k = 1; k <= 5; k = k + 1) {  
  console.log("k: " + k);  
}
```

k: 1

k: 2

k: 3

k: 4

k: 5

JS: condição de parada

- ❖ A condição de parada define quando o laço deve ser encerrado.
- ❖ Sempre que essa condição deixar de ser verdadeira, a repetição será interrompida.

```
for (let k = 1; k <= 5; k++) {  
  console.log("k: " + k);  
}
```

Fonte: autora, 2026

JS: condição de parada

- ❖ Todo loop deve possuir uma condição de parada adequada para garantir que a execução termine no momento esperado.

```
let i = 1;

while (i <= 5) {
  console.log("i: " + i);
  i++;
}
```

Fonte: autora, 2026

Tipos de loops

JS: while

- ❖ É utilizado quando não sabemos exatamente quantas repetições serão necessárias.
- ❖ Ele executa um bloco de código enquanto uma condição for verdadeira. Antes de cada repetição, a condição é verificada e quando ela se torna falsa, o laço é encerrado.

```
i = 1;
while (i <= 5) {
  console.log("i: " + i);
  i++;
}
```

i: 1
i: 2
i: 3
i: 4
i: 5

JS: do while

- ❖ O *do...while* é semelhante ao *while*, porém executa o **bloco de código** pelo menos uma vez antes de **verificar** a condição.
- ❖ Ele é indicado quando uma ação precisa ocorrer obrigatoriamente uma primeira vez, independentemente do resultado da condição.

```
let j = 1;  
  
do {  
    console.log("j: " + j);  
    j++;  
} while (j <= 5);
```

```
j: 1  
j: 2  
j: 3  
j: 4  
j: 5
```


JS: for

- ❖ É geralmente utilizado quando a **quantidade de repetições** é conhecida ou quando existe uma variável de controle.
- ❖ Sua estrutura reúne em uma única linha a **inicialização da variável**, a condição de repetição e a **atualização da variável de controle**, tornando-o muito utilizado para contagens e percorrer sequências de valores.

```
for (let k = 1; k <= 5; k++) {  
  console.log("k: " + k);  
}
```

Comandos para loops

JS: break

- ❖ O comando *break* é utilizado para **interromper imediatamente** a execução de um laço de repetição, mesmo que a **condição de parada** ainda não tenha sido atingida.
- ❖ Ele é útil quando encontramos o **resultado desejado** e não há necessidade de **continuar executando** as próximas repetições.

```
for (let k = 1; k <= 5; k++) {  
  if (k === 3) {  
    break;  
  }  
  console.log("k: " + k);  
}
```

k: 1
k: 2

Fonte: autora, 2026

JS: continue

- ❖ O continue interrompe apenas a iteração atual do laço e **passa para a próxima repetição**.
- ❖ Ele é utilizado quando desejamos ignorar uma **determinada condição** sem encerrar completamente o loop.

```
for (let k = 1; k <= 5; k++) {  
  if (k === 3) {  
    continue;  
  }  
  console.log("k: " + k);  
}
```

k: 1
k: 2
k: 4
k: 5

Fonte: autora, 2026

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Loops infinitos

JS: loop infinito

- ❖ Um loop infinito ocorre quando a **condição de parada nunca é atingida** ou quando a variável de controle **não é atualizada corretamente**.

```
while(1 == 1)
```

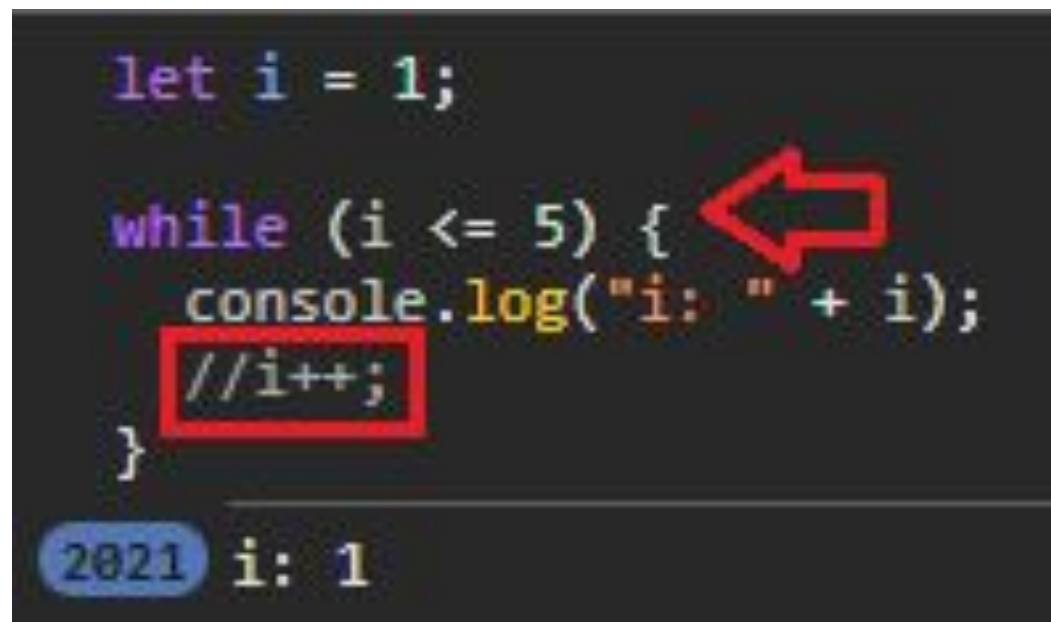
```
I'M STUCK IN A LOOP!  
I'M STUCK IN A LOOP!  
I'M STUCK IN A LOOP!  
I'M STUCK IN A LOOP!
```

JS: loop infinito

- ❖ O bloco de código continua sendo executado indefinidamente, podendo causar travamentos, consumo excessivo de recursos e impedir o funcionamento correto da aplicação.

```
let i = 1;
while (i <= 5) {
  console.log("i: " + i);
  //i++;
}
```

2021 i: 1



JS: loop infinito

- ❖ Para evitá-lo, é importante garantir que a **condição de parada** possa ser alcançada durante a **execução** do laço.

```
let i = 1;

while (i <= 5) {
  console.log("i: " + i);
  i++;
}
```

Loops aninhados

JS: loops aninhados

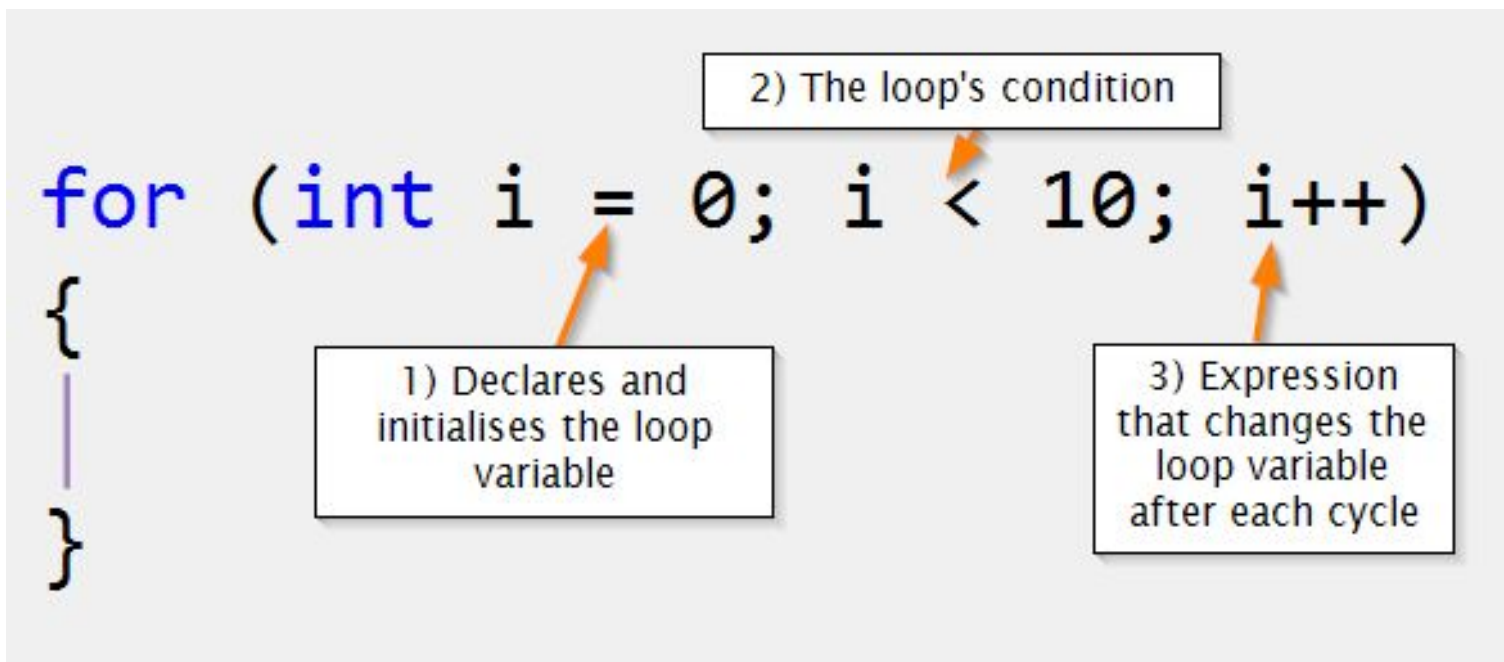
- ❖ São laços de repetição declarados **dentro de outros laços**. Eles são úteis quando precisamos trabalhar com **estruturas mais complexas**, como tabelas, matrizes ou combinações de elementos.
- ❖ Para cada repetição do **laço externo**, o **laço interno** é executado completamente antes que a próxima iteração do laço externo ocorra.

```
for (let a = 1; a <= 2; a++) {  
  for (let b = 1; b <= 3; b++) {  
    console.log("b: " + b);  
  }  
  console.log("a: " + a);  
}
```

Fonte: autora, 2026

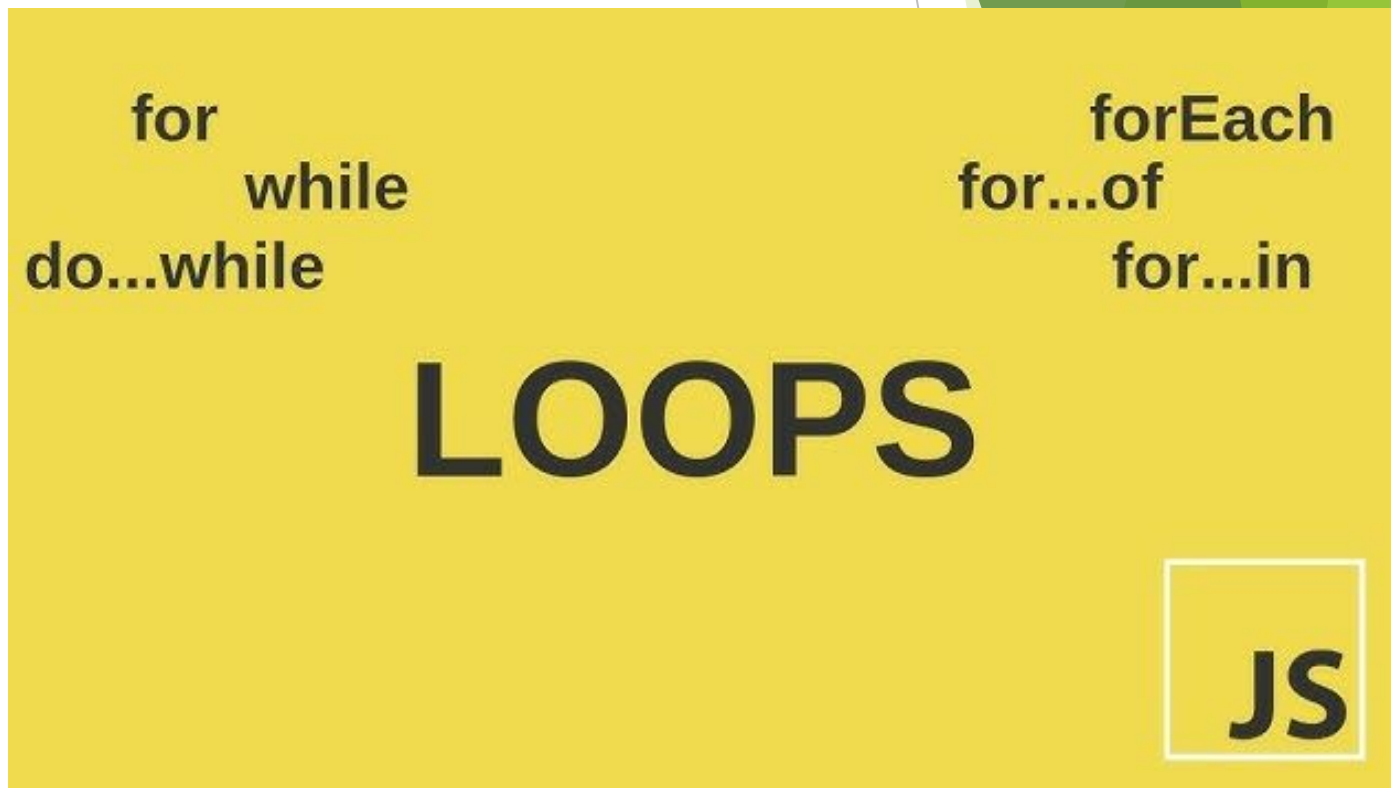
JS: qual loop utilizar?

- ❖ De forma geral, utilize *for* quando a **quantidade de repetições** for conhecida, *while* quando a **repetição** depender de uma condição cuja duração é desconhecida.



JS: qual loop utilizar?

- ❖ E *do...while* quando o bloco precisar ser executado ao menos uma vez antes da validação da condição.
- ❖ A escolha adequada torna o código mais legível e facilita sua manutenção.



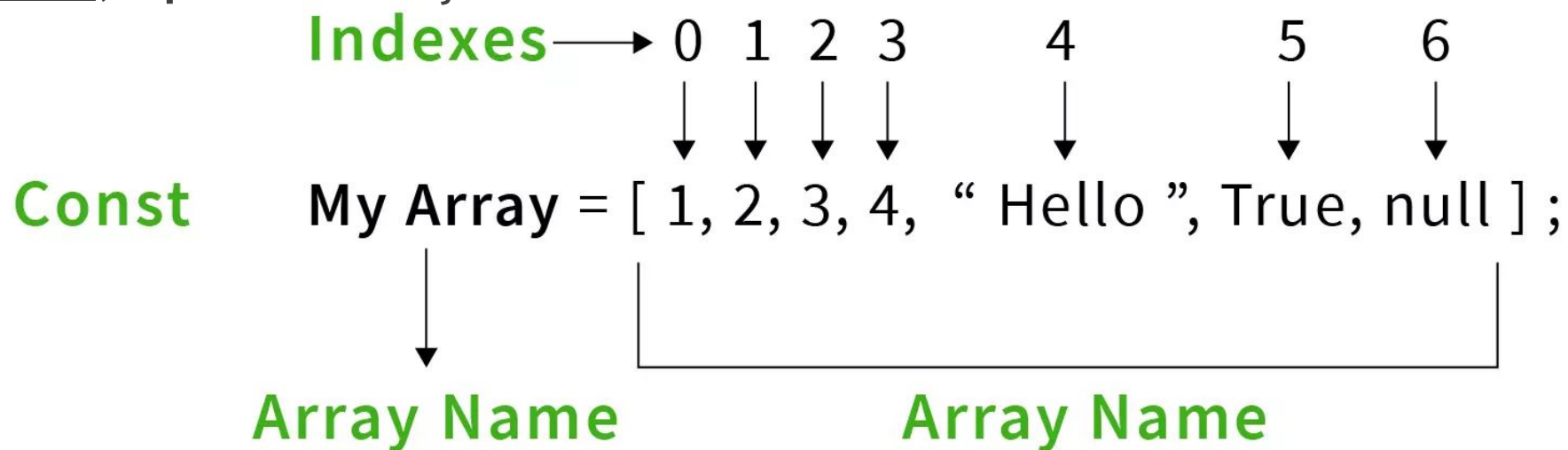
Fonte: Grey, 2025

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Array

JS: Arrays

- ❖ Um array é uma estrutura utilizada para **armazenar múltiplos valores** em uma única variável.
- ❖ Cada valor armazenado possui uma **posição**, chamada de índice, que começa em **zero**.



JS: Arrays

São muito utilizados para representar listas de dados, como:

- ❖ nomes de alunos
- ❖ produtos
- ❖ notas ou qualquer conjunto de informações relacionadas.

array declaration

const days = ["sun", "mon", "tues"];

array name

index 0

index 1

index 2

length = 3

JS: Arrays e Strings

- ❖ Uma string é uma sequência de caracteres utilizada para representar textos no JavaScript.
- ❖ Embora se comporte de forma parecida com um array em algumas situações (permitindo acessar caracteres por posição), uma string não é um array.

```
let languageNameString = "JavaScript";  
const technologiesArray = ["HTML", "CSS", "JavaScript"];  
console.log(languageNameString[0]);  
  
console.log(languageNameString.length);  
10  
  
console.log(technologiesArray[0]);  
HTML  
  
console.log(technologiesArray.length);  
3
```

JS: Arrays e Strings

- ❖ Uma *string* e um *array* possuem algumas **semelhanças**, ambos permitem acessar valores por posição utilizando índices.
- ❖ Porém, uma *string* representa um texto e seus caracteres **não podem ser alterados** individualmente.

```
let languageNameString = "JavaScript";  
languageNameString[0] = "T";  
console.log(languageNameString);  
  
JavaScript
```

JS: Arrays e Strings

- ❖ Já um array é uma **coleção de valores** que pode ser modificada livremente, permitindo adicionar, remover ou alterar elementos.

```
const technologiesArray = ["HTML", "CSS", "JavaScript"];
console.log(technologiesArray);

▶ (3) ['HTML', 'CSS', 'JavaScript']

technologiesArray[0] = "TypeScript";
console.log(technologiesArray);

▶ (3) ['TypeScript', 'CSS', 'JavaScript']
```

Métodos para Array

JS: métodos para Array

Método	Descrição	Exemplo	Resultado
push()	Adiciona um elemento ao final e retorna a quantidade total atualizada	[1,2,3,4].push(5);	5
pop()	Remove o último elemento e o retorna	[1,2,3,4].pop();	4
shift()	Remove o primeiro elemento e o retorna	[1,2,3,4].shift();	1
unshift()	Adiciona elemento no início e retorna a quantidade total atualizada	[1,2,3,4].unshift(0);	5
indexOf()	Verifica se existe no array um valor	[1,2,3,4].includes(2);	true
startsWith()	Retorna a posição (index) de um elemento	[1,2,3,4].indexOf(3);	2
find()	Retorna o primeiro elemento encontrado	[1,2,3,4,5].find(x => x > 3);	4
filter()	Filtra elementos	[1,2,3,4, 5].filter(x => x > 3);	[4, 5]

JS: métodos para Array

Método	Descrição	Exemplo	Resultado
map()	Transforma elementos	<code>[1,2,3,4].map(x => x * 2);</code>	<code>[2, 4, 6, 8]</code>
forEach()	Executa uma função para cada elemento	<code>[1,2,3,4].forEach(x => console.log(x));</code>	<code>1, 2, 3, 4</code>
some()	Verifica se algum atende à condição	<code>[1,2,3,4].some(x => x > 2);</code>	<code>true</code>
every()	Verifica se todos atendem à condição	<code>[1,2,3,4].every(x => x > 0);</code>	<code>true</code>
join()	Junta os elementos em uma string, e os separa pelo valor do join	<code>[1,2,3,4].join("-");</code>	<code>'1-2-3-4'</code>
slice()	Copia parte do array a partir dos índices indicado	<code>[1,2,3,4].slice(1,3);</code>	<code>[2, 3]</code>
splice()	Remove ou adiciona elementos	<code>[1,2,3,4].splice(2,3);</code>	<code>[3, 4]</code>

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The word "Object" is centered in a green, sans-serif font.

Object

JS: Object

- ❖ Um objeto é uma estrutura utilizada para **representar uma entidade** por meio de propriedades e valores.
- ❖ Enquanto um array organiza dados por posição (index), um **objeto** organiza dados por nome.

```
▶ {name: 'Sabrina', age: 40, technologies: Array(3)}  
▼ {name: 'Sabrina', age: 40, technologies: Array(3)}  
  age: 40  
  name: "Sabrina"  
  ▼ technologies: Array(3)  
    0: "HTML"  
    1: "CSS"  
    2: "JavaScript"  
    length: 3  
    ▶ [[Prototype]]: Array(0)  
    ▶ [[Prototype]]: Object
```

Fonte: autora, 2026

JS: Object

- ❖ É ideal para armazenar informações relacionadas a um mesmo elemento, como os dados de um aluno, produto ou usuário.

```
const labrador = {  
  name: "Rex",  
  age: 3,  
  breed: "Labrador",  
};  
console.log(labrador.name);  
Rex  
console.log(labrador.age);  
3
```

```
const caramelo = {  
  name: "Caramelo",  
  age: 1,  
  breed: "Desconhecido",  
};  
console.log(caramelo.name);  
Caramelo  
console.log(caramelo["breed"]);  
Desconhecido
```

JS: verificação de arrays

- ❖ Os *arrays* são um **tipo especial** de objeto. Por isso, ao utilizar o operador *typeof*, o resultado retornado para um array é "*object*" e não "*array*".
- ❖ Para **verificar corretamente** se um valor é um *array*, deve-se utilizar o método *Array.isArray()*, que retorna *true* para arrays e *false* para qualquer outro tipo de valor.

```
const numbers = [10, 20, 30];  
console.log(typeof numbers);  
object  
console.log(Array.isArray(numbers));  
true
```

Métodos para Object

JS: métodos para Object

Método	Descrição	Exemplo	Resultado
Object.keys()	Retorna as chaves do objeto	Object.keys({nome:"Ana"})	["nome"]
Object.values()	Retorna os valores do objeto	Object.values({nome:"Ana"})	["Ana"]
Object.entries()	Retorna chave e valor e uma lista	Object.entries({nome:"Ana"})	['nome', 'Ana']
Object.assign()	Copia propriedades	Object.assign({nome:"Ana"}, {})	{nome: 'Ana'}
Object.freeze()	Impede alterações e retorna o objeto	Object.freeze({nome:"Ana"})	{nome: 'Ana'}
Object.seal();	impede adicionar ou remover propriedades, mas permitindo alterar os valores das propriedades já existentes	Object.seal({nome:"Ana"})	{nome: 'Ana'}
Object.hasOwnProperty()	Verifica se possui uma propriedade	({nome:"Ana"}).hasOwnProperty("nome")	true

JS: Object.freeze() x Object.seal()

- ❖ *Object.freeze()* e *Object.seal()* são métodos para **aumentar o controle sobre objetos** em JavaScript, porém possuem comportamentos diferentes.
- ❖ Quando um objeto é congelado com *Object.freeze()*, **nenhuma alteração é permitida**: não é possível adicionar novas propriedades, remover propriedades existentes ou modificar os valores já armazenados.

```
> const person = {  
  name: "Sabrina",  
  age: 40,  
  technologies: ["HTML", "CSS", "JavaScript"],  
  
  talk(text) {  
    console.log(text);  
  },  
};  
Object.freeze(person);  
person.nationality = "Brasileira";  
person.name = "Sabrina B";  
console.log(person);  
  
▼ {name: 'Sabrina', age: 40, technologies: Array(3), talk: f} ⓘ  
  age: 40  
  name: "Sabrina"  
  ► talk: f talk(text)  
  ► technologies: (3) ['HTML', 'CSS', 'JavaScript']  
  ► [[Prototype]]: Object
```


JS: Object.freeze() x Object.seal()

- ❖ Já o *Object.seal()* oferece uma **proteção mais flexível**, impedindo apenas a **adição e remoção de propriedades**, mas permitindo que os valores das **propriedades existentes** sejam alterados.

```
const person = {
  name: "Sabrina",
  age: 40,
  technologies: ["HTML", "CSS", "JavaScript"],

  talk(text) {
    console.log(text);
  },
};

Object.seal(person);
person.nationality = "Brasileira";
person.name = "Sabrina B";
console.log(person);
```

▼ {name: 'Sabrina B', age: 40, technologies: Array(3), talk: f}

- age: 40
- name: "Sabrina B"
- talk: f talk(text)
- technologies: (3) ['HTML', 'CSS', 'JavaScript']
- [[Prototype]]: Object

Função x método

JS: Função x Método

- ❖ Um método é uma função que pertence a um objeto.
- ❖ A principal diferença é que a função existe de forma independente, enquanto o método está associado a um objeto e normalmente atua sobre os dados desse objeto.

```
const person = {  
  name: "Sabrina",  
  age: 40,  
  technologies: ["HTML", "CSS", "JavaScript"],  
  talk(text) {  
    console.log(text);  
  },  
};  
console.log(person);  
▶ {name: 'Sabrina', age: 40, technologies: Array(3), talk:  
person.talk("oi");  
oi
```

Fonte: autora, 2026

JS: Função x Método

❖ Embora ambos executam blocos de código, uma função é chamada diretamente pelo seu nome, enquanto um método é chamado a partir de um objeto utilizando a notação de ponto (.).

❖ Em outras palavras, todo método é uma função, mas nem toda função é um método.

```
const person = {  
  name: "Sabrina",  
  age: 40,  
  technologies: ["HTML", "CSS", "JavaScript"],  
  talk(text) {  
    console.log(text);  
  },  
};  
person.talk("oi");
```

oi

```
function talk(text) {  
  console.log(text);  
}  
talk("oi");
```

oi

JS: Função x Método

❖ Embora ambos executam blocos de código, uma função é chamada diretamente pelo seu nome, enquanto um método é chamado a partir de um objeto utilizando a notação de ponto (.).

❖ Em outras palavras, todo método é uma função, mas nem toda função é um método.

```
const person = {  
  name: "Sabrina",  
  age: 40,  
  technologies: ["HTML", "CSS", "JavaScript"],  
  talk(text) {  
    console.log(text);  
  },  
};  
person.talk("oi");
```

oi

```
function talk(text) {  
  console.log(text);  
}  
talk("oi");
```

oi

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The word "Date" is centered in a large, green, sans-serif font.

Date

JS: Date

- ❖ O objeto *Date* é utilizado para trabalhar com **datas e horários** no JavaScript.
- ❖ Com ele é possível obter a data atual, extrair informações como dia, mês e ano, calcular diferenças entre datas e formatar datas para exibição ao usuário.

```
const currentDate = new Date();
console.log(currentDate);

Tue Jun 16 2026 18:29:47 GMT-0300 (Horário Padrão de Brasília)

console.log(currentDate.getFullYear());

2026

console.log(currentDate.getMonth());

5

console.log(currentDate.getDate());

16
```

Fonte: autora, 2026

JS: Date

- ❖ É amplamente utilizado em sistemas que precisam registrar eventos, prazos, datas de cadastro ou horários.

```
const currentDate = new Date();
console.log(currentDate);
Tue Jun 16 2026 18:32:51 GMT-0300 (Horário Padrão de Brasília)
console.log(currentDate.getHours());
18
console.log(currentDate.getMinutes());
32
console.log(currentDate.toLocaleDateString("pt-BR"));
16/06/2026
console.log(currentDate.toLocaleTimeString("pt-BR"));
18:32:51
```

Fonte: autora, 2026

Métodos para Date

JS: métodos para Date

Método	Descrição	Exemplo	Resultado
getFullYear()	Retorna o ano	new Date().getFullYear()	2026
getMonth()	Retorna o mês (0 a 11)	new Date().getMonth()	5
getDate()	Retorna o dia do mês	new Date().getDate()	12
getHours()	Retorna a hora	new Date().getHours()	14
getMinutes()	Retorna os minutos	new Date().getMinutes()	45
toLocaleDateString()	Formata a data	new Date().toLocaleDateString("pt-BR")	"12/06/2026"
toLocaleTimeString()	Formata a hora	new Date().toLocaleTimeString("pt-BR")	"14:30:45"

Operações com datas

JS: Date

- ❖ O objeto *Date* trabalha internamente com milissegundos e por isso para as datas que vemos formatadas (dia, mês, ano, hora), existe um **valor numérico contínuo** representando o tempo em milissegundos.
- ❖ Todas as **operações com datas** são feitas de forma matemática, somando ou subtraindo milissegundos para avançar ou retroceder no tempo.

```
const currentDate = new Date();  
console.log("currentDate:", currentDate);  
currentDate: Tue Jun 23 2026 15:06:16 GMT-0300 (Horário Padrão de Brasília)  
const someDate = new Date("2026-06-23");  
console.log("someDate:", someDate);  
someDate: Mon Jun 22 2026 21:00:00 GMT-0300 (Horário Padrão de Brasília)
```

JS: Date

- ❖ É por isso que conseguimos facilmente adicionar ou remover dias, meses, horas, minutos ou segundos usando métodos como *setDate()*, *setMonth()*, *setHours()*...
- ❖ Trabalhar com datas no *JavaScript* não é manipular “texto de data”, mas sim realizar operações numéricas representada em milissegundos.

```
const add7Days = new Date(someDate);
add7Days.setDate(add7Days.getDate() + 7);
console.log("+7 dias:", add7Days.toLocaleDateString("pt-BR"));
+7 dias: 29/06/2026

const add3Hours = new Date(someDate);
add3Hours.setHours(add3Hours.getHours() + 3);
console.log("+3 horas:", add3Hours.toLocaleTimeString("pt-BR"));
+3 horas: 00:00:00

const add50Sec = new Date(someDate);
add50Sec.setSeconds(add50Sec.getSeconds() + 50);
console.log("+50 segundos:", add50Sec.toLocaleTimeString("pt-BR"));
+50 segundos: 21:00:50
```

Fonte: autora, 2026

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Vídeos complementares

- Assista os vídeos abaixo para revisar o conteúdo dessa aula.



Exercício

Exercício 1

- ❖ Vamos montar um script que analisa a **situação** de vários acadêmicos armazenados em um *array de objetos*.
- ❖ Primeiro, devemos percorrer todos os acadêmicos utilizando um **laço de repetição**.
- ❖ Depois, devemos verificar se a média de notas de **cada acadêmico** é igual ou maior que 6.
- ❖ Ao final, deverá exibir **quais acadêmicos** foram aprovados e quais foram **reprovados** com base na média.

Exercícios

- ❖ Ambos os exercícios percorrerão o Array de objetos abaixo:

```
const academicos = [  
  {  
    nome: "Ana",  
    mediaDeNotas: 7,  
    totalDeFaltas: 19,  
  },  
  {  
    nome: "Carlos",  
    mediaDeNotas: 5,  
    totalDeFaltas: 18,  
  },  
  {  
    nome: "João",  
    mediaDeNotas: 9,  
    totalDeFaltas: 6,  
  },  
];
```

Objeto 1

Objeto 2

Objeto 3

Exercício 1

```
// Exercício 1
{
  console.log("\nExercício 1\n");
  for (let i = 0; i < academicos.length; i++) {
    const academico = academicos[i];

    if (academico.mediaDeNotas >= mediaDeNotasMinimaParaAprovacao) {
      console.log(`${academico.nome} aprovado`);
    } else {
      console.log(`${academico.nome} reprovado`);
    }
  }
}
```

Exercício 1

Ana aprovado

Carlos reprovado

João aprovado

Exercício 2

- ❖ Vamos evoluir o script de **aprovação acadêmica** utilizando *arrays*, *objetos*, *funções* e *laços de repetição*.
- ❖ Primeiro, devemos **percorrer todos os acadêmicos** e obter suas informações de nota e frequência.
- ❖ Depois, devemos **verificar individualmente** se cada acadêmico foi aprovado por nota e se possui **frequência mínima de 75%**.
- ❖ A aprovação final deverá ser definida pela aprovação por **nota E por frequência** para cada acadêmico.

Exercício 2

❖ Parte 1:

```
function verificacaoAprovacaoPorNotas(
  academico,
  mediaDeNotasMinimaParaAprovacao,
) {
  const mediaDeNotas = academico.mediaDeNotas || 0;
  if (mediaDeNotas >= mediaDeNotasMinimaParaAprovacao) {
    return true;
  } else {
    console.log(
      `Acadêmico(a) ${academico.nome} reprovado(a) por média de notas de ${parseFloat(mediaDeNotas)}`,
    );
    return false;
  }
}
```

Fonte: autora, 2026

Exercício 2

❖ Parte 2:

```
function verificacaoAprovacaoPorFrequencia() {  
  const porcentagemFrequenciaMinimaParaAprovacao = 75;  
  const totalAulas = 72;  
  const totalDeFaltas = Number(  
    prompt("Digite o total de faltas do acadêmico no semestre: "),  
  );  
  
  // Calcula a porcentagem de frequência do acadêmico.  
  // 1. Subtrai as faltas do total de aulas para descobrir quantas aulas foram frequentadas.  
  // 2. Multiplica o resultado por 100.  
  // 3. Divide pelo total de aulas para obter a porcentagem de frequência.  
  const porcentagemFrequenciaTotal =  
    ((totalAulas - totalDeFaltas) * 100) / totalAulas;  
  
  // Verifica se a porcentagem de frequência do acadêmico é maior ou igual a porcentagem mínima de frequência para aprovação  
  if (  
    porcentagemFrequenciaTotal >= porcentagemFrequenciaMinimaParaAprovacao  
  ) {  
    return true;  
  } else {  
    console.log(  
      `Reprovado por frequência. Frequência total: ${parseInt(porcentagemFrequenciaTotal)}%`,  
    );  
    return false; // reprovado: as faltas devem ser menor ou igual a 25% do total de aulas  
  }  
}
```


Exercício 2

❖ Parte 3:

```
function verificacaoDeAcademicoAprovado(
  academico,
  mediaDeNotasMinimaParaAprovacao,
) {
  if (
    verificacaoAprovacaoPorNotas(
      academico,
      mediaDeNotasMinimaParaAprovacao,
    ) &&
    verificacaoAprovacaoPorFrequencia(academico)
  ) {
    console.log(`Acadêmico(a) ${academico.nome} aprovado(a).`);
    return true;
  } else {
    return false;
  }
}
```

Exercício 2

- ❖ Parte 4: chamada da função principal no laço de repetição:

```
console.log("\nExercício 2\n");  
for (let i = 0; i < academicos.length; i++) {  
    const academico = academicos[i];  
  
    verificacaoDeAcademicoAprovado(academico, mediaDeNotasMinimaParaAprovacao);  
}
```

Exercício 2

Acadêmico(a) Ana reprovado(a) por frequência de 73%

Acadêmico(a) Carlos reprovado(a) por média de notas de 5

Acadêmico(a) João aprovado(a).

Exercícios

- ❖ Confira o código completo desses exercícios no link abaixo:
- ❖ [Exemplo](#)

Exercício 3

- ❖ Agora teste você em seu navegador os scripts vistos nessa aula.
- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos

Referência da aula

- ❖ LEBEC, Gabriel. JavaScript: a history for beginners. Course Report, 2019. Disponível em: <https://www.coursereport.com/blog/history-of-javascript>. Acesso em: 23 mar. 2026.
- ❖ MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.119. ISBN 9788582601969. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/>. Acesso em: 11 mai. 2026.
<https://www.task.com.br/blog/o-que-e-https/>

Referência da aula

- ❖ SCALER TOPICS. JavaScript Array. Scaler, [s.d.]. Disponível em: <https://www.scaler.com/topics/javascript/javascript-array/>. Acesso em: 30 mar. 2026.



univille.br